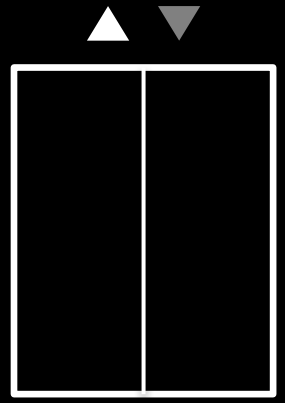
# Card-Access Hotel Elevator



STM32 Nucleo-F411RE Based On Embedded System

**Team 15 | 정명훈 , 이기광**

2026. 11. 09

# Contents


01 Overview

02 Functions & Demo Video

03 BOM (Bill of Materials)

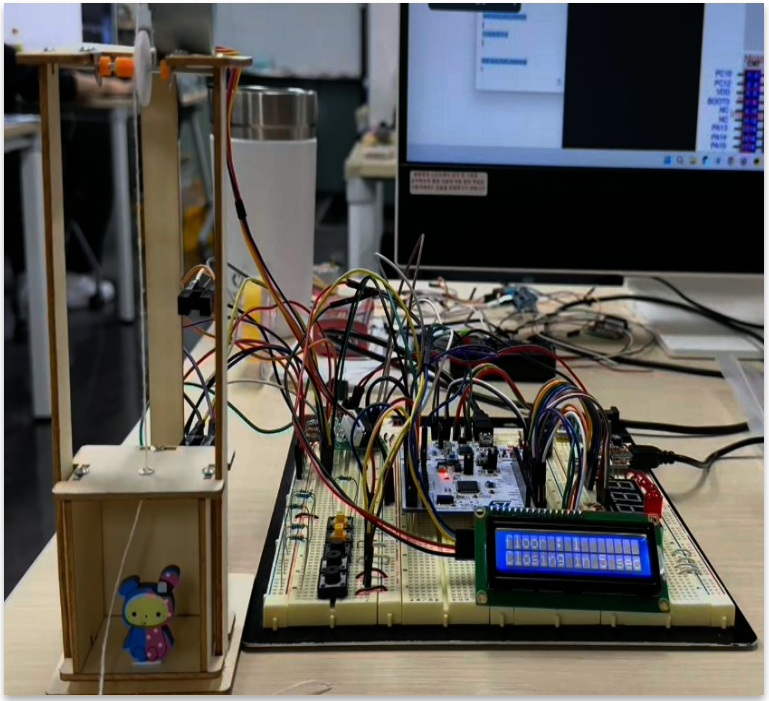
04 H · W Structure

05 S · W Structure

06 Troubleshooting

07 Conclusion & Discussion

# Overview



## Summary

카드에 등록된 층으로만 갈 수 있는
3 층 카지노형 엘리베이터 .
스텝모터로 엘레베이터를 움직이고
포토인터럽터로 층을 감지하며 ,
PC 와  UART 로 카드 태그와 문 동작을
주고받음 .

# Functions

## Card Access

- 카드 DB: 손님 (2 층 ), 직원 ( 전 층 )
- 외부 리더 태그 = 그 층 호출
- 손님 목적층은 태그 시점에 자동 등록 (1 층 ↔ 2 층 )
- 권한 없는 카드는 응답 없이 무시

## Master Mode

- 직원 카드로 내부 리더 태그 시 진입
- 5 초 안에 층 버튼 1 회 입력
- 입력 즉시 일반 모드 복귀

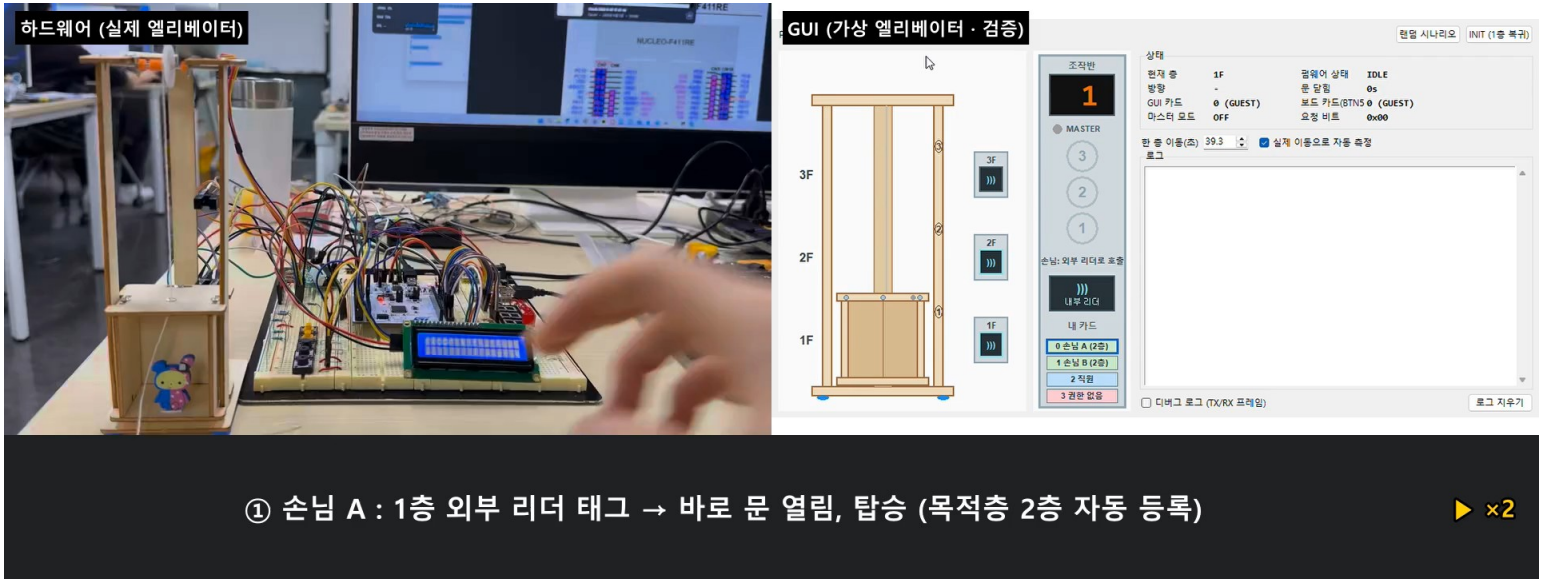
## Run & Door

- 방향 우선
- 해당층 요청 한번에 처리
- 포토인터럽터로 층 도착 정지
- 문 열림 5 초 카운트를 LCD 에 표시
- 문 닫힘 신호 전에는 출발 금지

### Button / LCD / Buzzer

**Button** BTN1~3(PC0~PC2) 1~3 층 외부 리더 , 마스터 모드에서는 층 버튼 · BTN4(PC3) 내부 리더 · BTN5(PB0) 카드 선택
**LCD** 1 행 floor : n · 2 행 Closing in n sec ( 문 열림 중 )
**Buzzer** 버튼과 GUI 입력마다 G6 → C7 두 음 (40 ms + 60 ms)

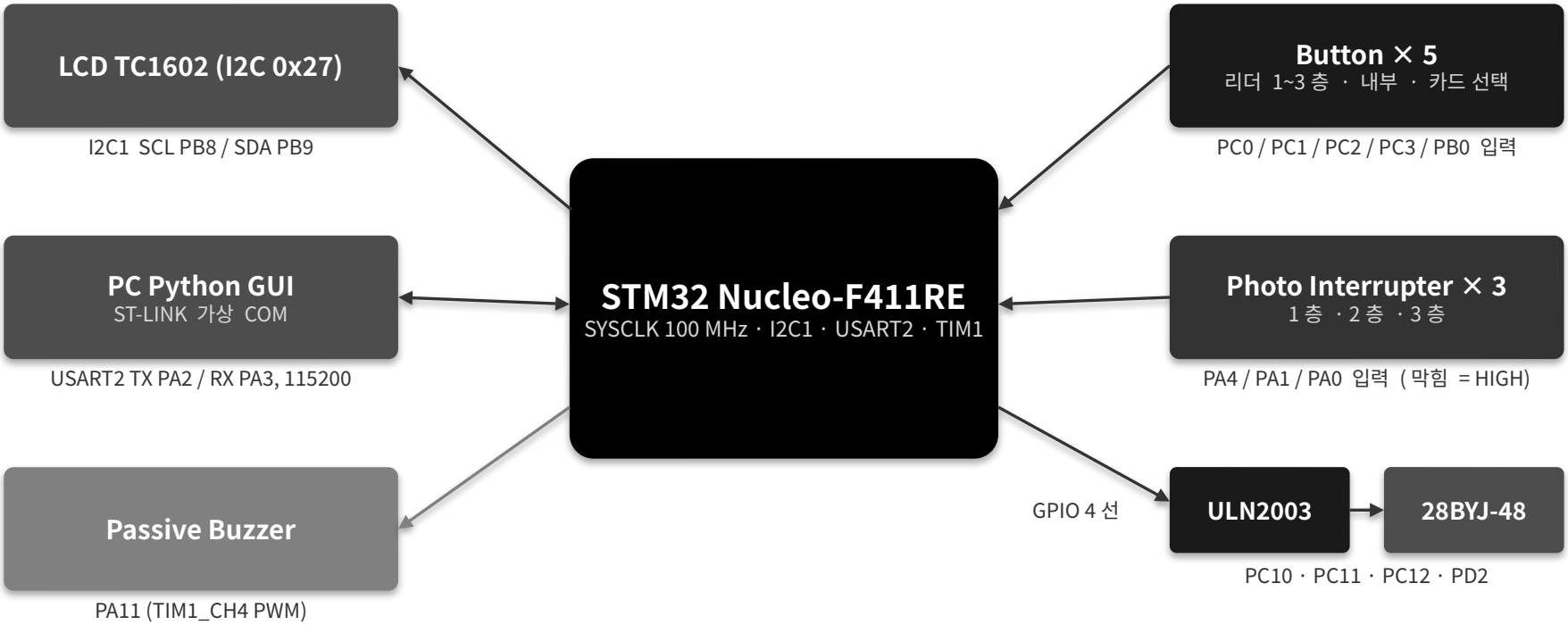
# Demo Video


하드웨어 (실제 엘리베이터)
GUI (가상 엘리베이터 · 검증)
3F
2F
1F
조작반
1
MASTER
내 카드
0 손님 A (2층)
1 손님 B (2층)
2 직원
3 권한 없음
상태
현재 층 1F
방향 -
GUI 카드 0 (GUEST)
마스터 모드 OFF
컴퓨어 상태 IDLE
문 닫힘 0s
보드 카드(BTN5 0 (GUEST)
요청 비트 0x00
로그
디버그 로그 (TX/RX 프레임)
로그 지우기
① 손님 A : 1층 외부 리더 태그 → 바로 문 열림, 탑승 (목적층 2층 자동 등록)
▶ ×2

# BOM (Bill of Materials)

| No | Part | Model / Spec | Qty | Usage / Interface |
|---|---|---|---|---|
| 1 | MCU 보드 | STM32 Nucleo-F411RE<br>(Cortex-M4, 100 MHz) | 1 | 메인 제어 , ST-LINK 디버깅과 가상 COM |
| 2 | Step Motor | 28BYJ-48 (5V) | 1 | 엘레베이터 승강 , 하강 |
| 3 | 모터 드라이버 | ULN2003 드라이버 보드 | 1 | GPIO 4 선 → 모터 코일 구동 |
| 4 | 포토인터럽터 | 슬롯형 (LED 330Ω, 풀업 4.7kΩ) | 3 | 층 감지 , 판이 막으면 HIGH |
| 5 | LCD | TC1602A-01T 16x2 (I2C) | 1 | I2C1 ( 주소 0x27), 현재 층과 문 카운트 |
| 6 | Buzzer | Passive Buzzer | 1 | TIM1 CH4 PWM, 버튼음 |
| 7 | Tact Switch | Push Button | 5 | 외부 리더 3, 내부 리더 1, 카드 선택 1 |
| 8 | PC | Python GUI | 1 | USART2 115200, 가상 엘리베이터와 문 |

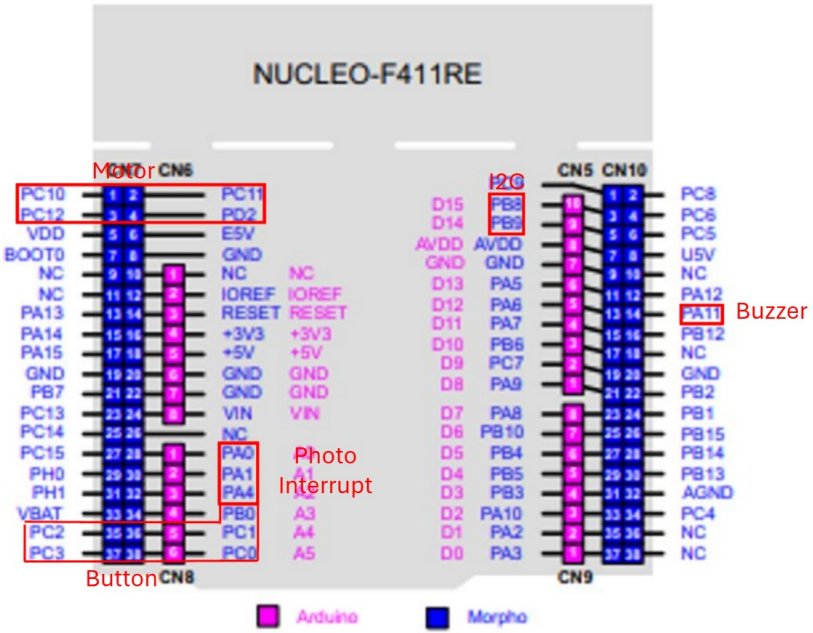
# H·W Structure



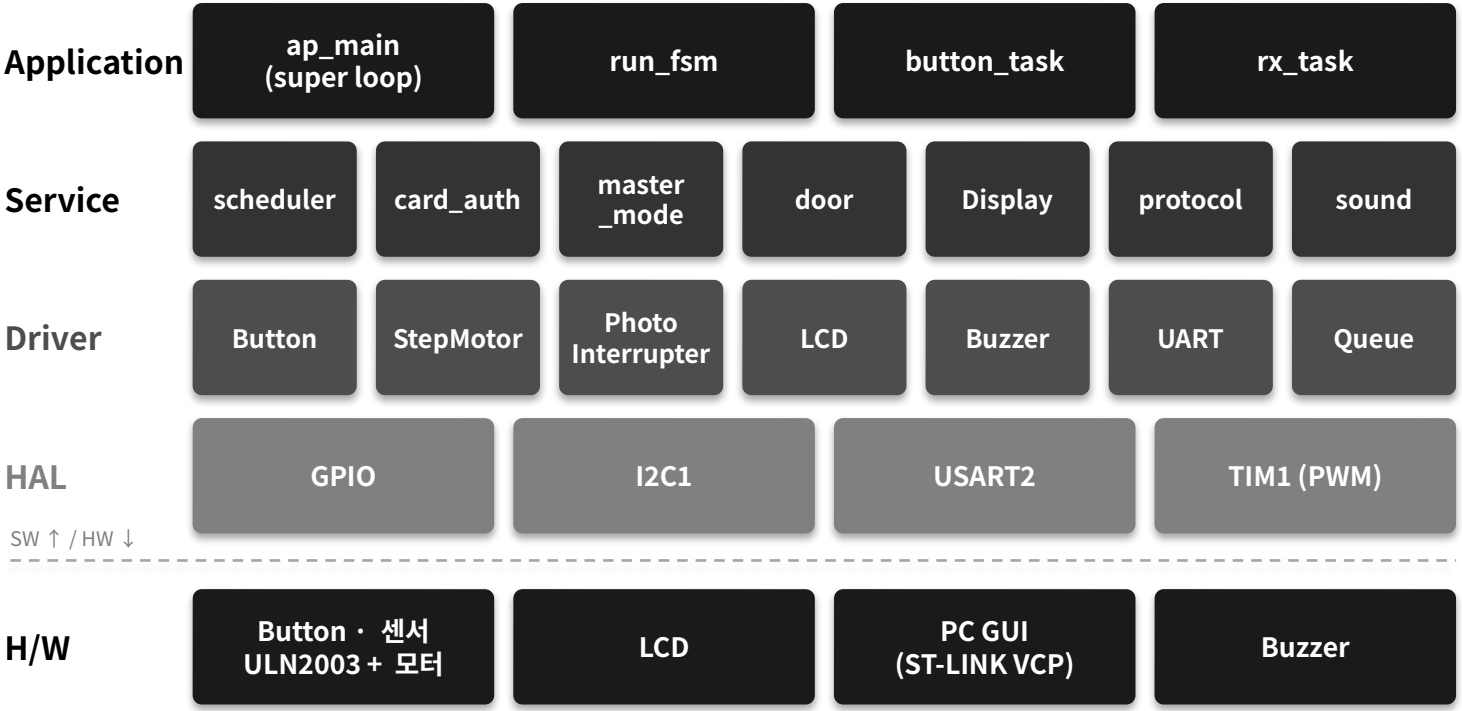
# Pin Map (Nucleo-F411RE)

| Function | Signal | Pin | Config |
|---|---|---|---|
| 1. I2C1 (LCD) | SCL / SDA | PB8 / PB9 | I2C |
| 2. USART2 (GUI) | TX / RX | PA2 / PA3 | 115200 8N1 |
| 3. 외부 리더 1~3 층 | btn[0~2] | PC0, PC1, PC2 | GPIO Input |
| 4. 내부 리더 | btn[3] | PC3 | GPIO Input |
| 5. 카드 선택 | btn[4] | PB0 | GPIO Input |
| 6. 층 센서 1~3 층 | sensor[0~2] | PA4, PA1, PA0 | GPIO Input |
| 7. Motor IN1~4 | motor_pin | PC10, PC11, PC12, PD2 | GPIO Output |
| 8. Buzzer | htim1 CH4 | PA11 | TIM1 PWM |

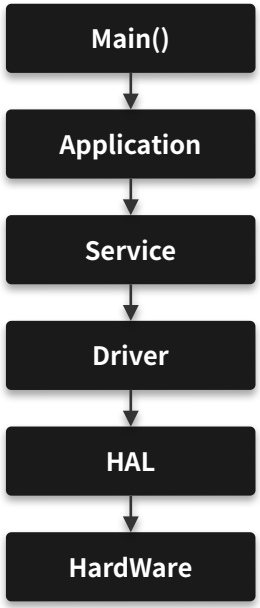
**Figure 18. NUCLEO-F411RE**



# S·W Structure (S·W Stack)




**Call Flow**




caller → callee
상위 계층 → 하위 계층

# Ap_main & Timing

## Motor 3 ms

MOTOR_TICK_MS 3

motor_task()
3 ms 마다 한 스텝 ( 하프스텝 8 패턴 )
한 층 구간 30000 스텝 안에 센서를 못 찾으면 정지

## Door 5 s

door_start() / door_getSec()

DOOR_OPENED 수신 후 시작
LCD 에 5 → 1 초 표시
같은 층 호출이면 다시 5 초

## Master 5 s

master_mode_is_active()

내부 리더 태그 후 시작
층 버튼 1 회 입력 또는 5 초 경과로 종료
문 5 초와 따로 동작

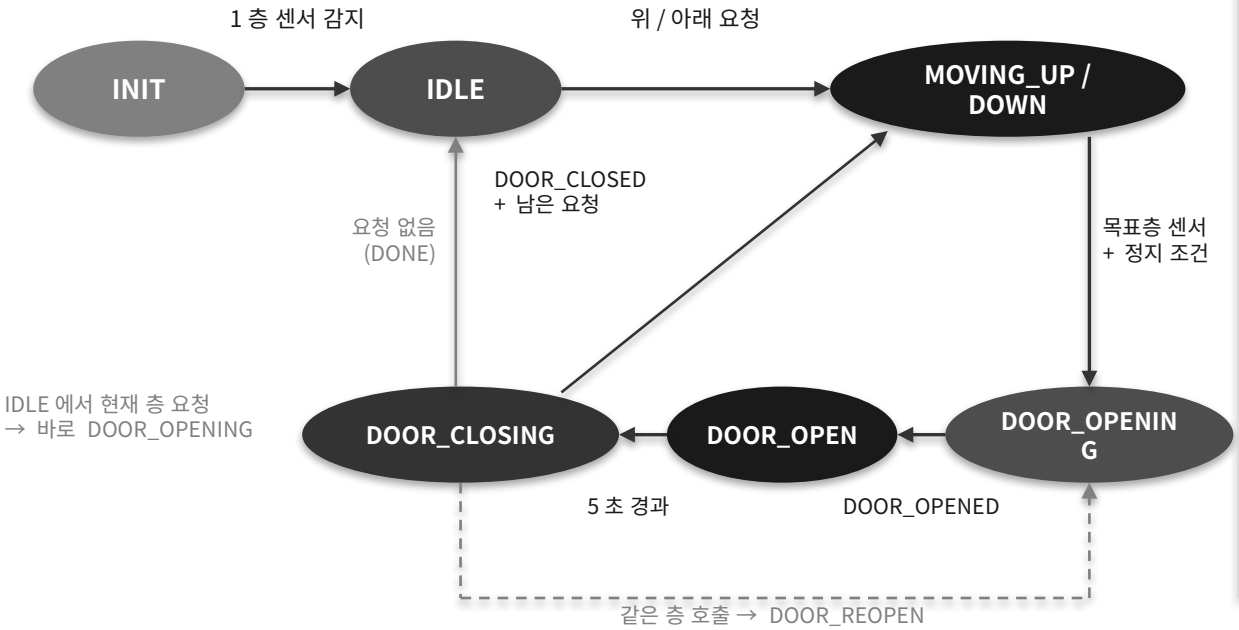
**ap_main**

button_task() → rx_task() → run_fsm() → Disp_LCD() → sound_task() 반복
모든 시간은 HAL_GetTick() 비교로 처리 , FSM 안에 delay 없음 → 이동 중이나 문이 열린 동안에도 카드와 GUI 입력 처리
UART 수신 인터럽트는 프레임을 수신 큐 (8 칸 ) 에 넣기만 하고 , 처리는 rx_task() 가 ap_main 에서 함

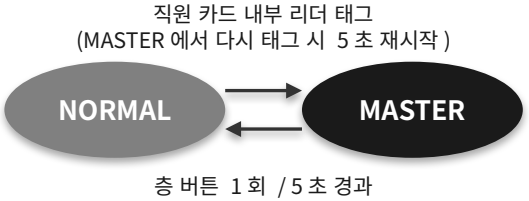
# State Machine (FSM)

2-process FSM: run_next_state() 가 다음 상태만 계산하고 , run_enter() 와 run_hold() 가 동작을 실행

## Run FSM (run_fsm)



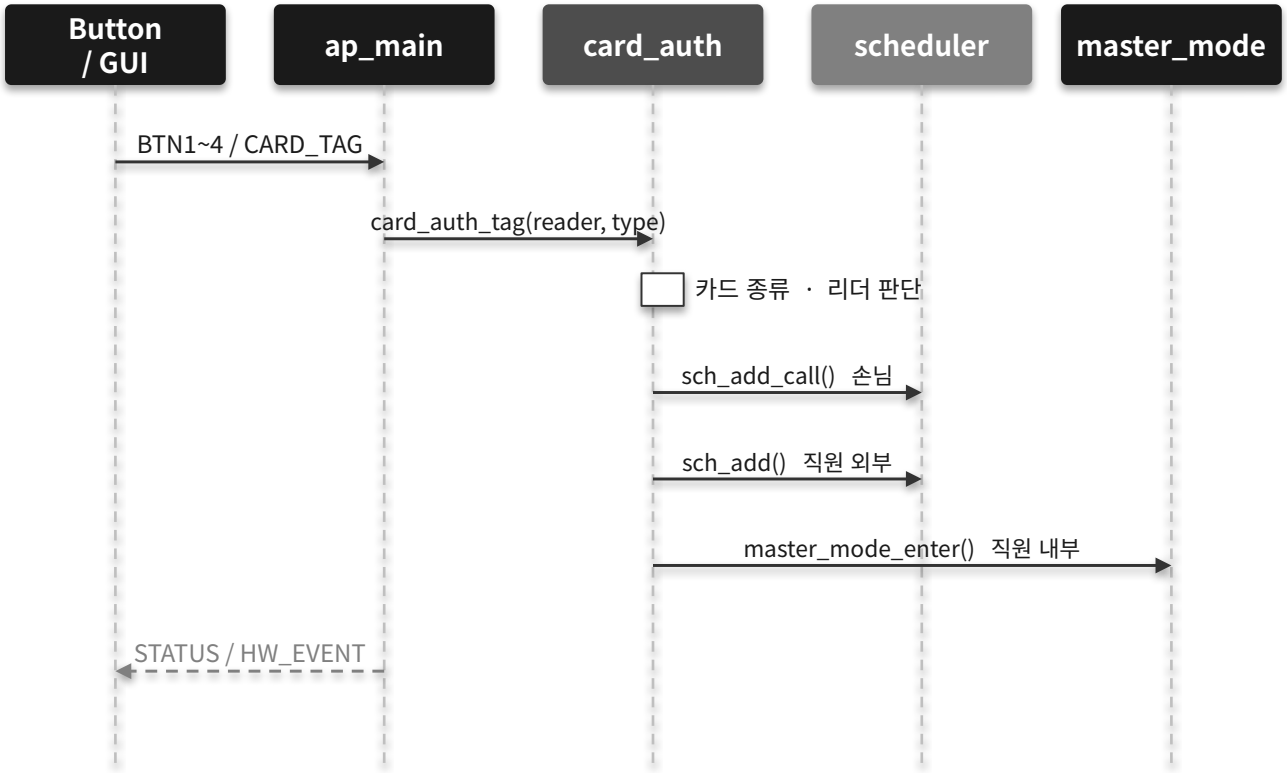

## Master Mode (master_mode)




### bool active

- 상태가 2 개라 변수 하나로 구현
- 운행 FSM 과 따로 돌고 서로 기다리지 않음
- 서 있는 층을 누르면 요청 없이 복귀

# ① Card Tag: Request Register




- **Card DB:** 손님 , 직원 , 권한 없음 3 장 . BTN5 로 순환
- **손님 :** 1 층 리더면 2 층 , 2 층 리더면 1 층을 대기 목적층으로 보관
- **탑승 시점 등록 :** 그 호출을 처리하는 순간 sch_serve() 가 목적층 비트로 옮김
- **직원 :** 외부 리더는 호출만 , 내부 리더는 마스터 모드
- **권한 없음 :** 무시 , NACK 없음

# ② Scheduler: Direction-First (SCAN)

**Request Register (8 bit, 순서 없음 )**

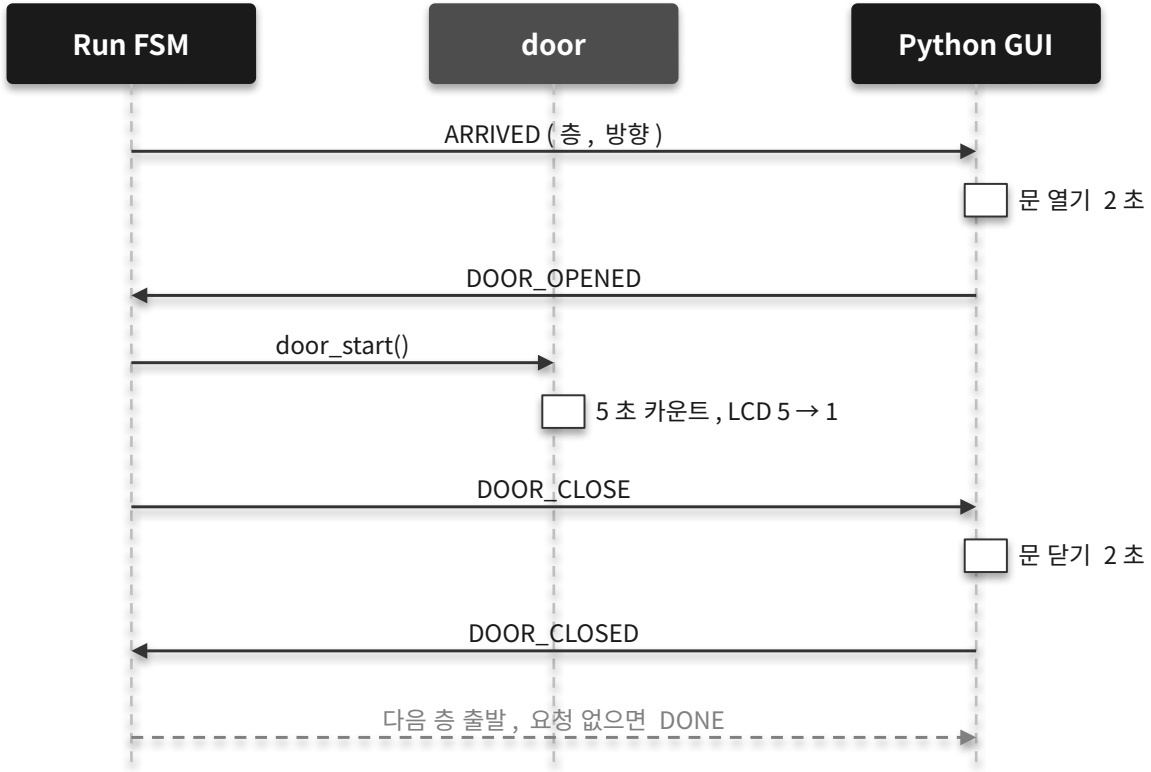
| bit6 | bit5 | bit4 | bit3 | bit2 | bit1 | bit0 |
|---|---|---|---|---|---|---|
| 3F 목적층 | 2F 목적층 | 1F 목적층 | 3F 하행 | 2F 방향무관 | 2F 하행 | 1F 상행 |

호출 (Hall Call) 은 bit0~3, 목적층 (Car Call) 은 bit4~6

| 시점 | 위치 / 방향 | 남은 요청 | 판단 |
|---|---|---|---|
| t0 | 1 층 정지 | 2 층 하행 | 상승 시작 |
| t1 | 상승 중 | + 3 층 직원 | 3 층 호출 추가 |
| t2 | 2 층 , 상승 | 2 층 하행 , 3 층 | 위에 요청 → 통과 |
| t3 | 3 층 정지 | 2 층 하행 | 하강으로 전환 |
| t4 | 2 층 정지 | 1 층 목적층 | 손님 목적층 등록 |

- **sch_should_stop():** 그 층 요청이 2 층 하행 호출 하나뿐이고 , 상승 중이며 위에 요청이 남아 있으면 통과 . 나머지는 정지
- **sch_serve():** 처리한 비트를 지우고 , 지운 호출에 묶인 대기 목적층을 등록
- **sch_next_dir():** 같은 방향 앞에 요청이 있으면 유지 , 없으면 반대 , 둘 다 없으면 NONE
- **시나리오 1 ( 왼쪽 표 ):** 2 층 손님 호출 뒤 3 층 직원 호출 . 2 층을 지나쳐 [3, 2, 1] 순서

# ③ Door: GUI Handshake




- **인터록 :** DOOR_CLOSED 를 받기 전에는 출발하지 않음
- **문은 GUI 만 :** STM32 는 시간만 세고 , 문 애니메이션은 GUI 가 맡음
- **재시작 :** 문이 열린 동안 같은 층 호출이 오면 5 초를 다시 셈
- **재열림 :** 닫히는 중 같은 층 호출이면 DOOR_REOPEN, GUI 는 다시 엶

# ④ UART Protocol: 8-Byte Frame




CHK = (LEN + CMD + P0 + P1 + P2 + P3) & 0xFF ( 덧셈 )
예 ) 3 층 리더에 직원 카드 : AA 05 01 03 02 00 00 0B

| CMD | GUI → STM32 | Payload |
|---|---|---|
| 0x01 | CARD_TAG | 리더 , 카드 종류 |
| 0x02 | DEST_BTN | 목적층 |
| 0x03 | DOOR_OPENED | 층 |
| 0x04 | DOOR_CLOSED | 층 |
| 0x05 | STATUS_REQ | - |
| 0x06 | INIT | - |

| CMD | STM32 → GUI | Payload |
|---|---|---|
| 0x81 | STATUS | 층 , 방향 , 상태 , 요청 |
| 0x82 | ARRIVED | 층 , 방향 |
| 0x83 | DOOR_CLOSE | 층 |
| 0x84 | DOOR_REOPEN | 층 |
| 0x85 | HW_EVENT | 버튼 , 카드 , 마스터 |
| 0x86 | DONE | 층 , 요청 |

- **수신 :** 인터럽트에서 1 바이트씩 받아 프레임을 맞추고 큐에 넣음
- **오류 :** SOF, LEN, CHK 가 틀리면 버림
- **응답 :** GUI 명령마다 STATUS 1 개

# Troubleshooting: Floor Sensor Not Detected

## Problem

- 층 센서가 막혀도 엘레베이터가 멈추지 않고 계속 이동
- 로직 아날라이저에서는 센서 신호가 바뀌는데 , 보드가 읽은 값은 그대로

## Cause

- PC14, PC15 는 보드 기본 설정에서 LSE 크리스털용이라 헤더와 끊겨 있음 (UM1724)
- PC13 은 USER 버튼과 연결

## Solution

- 센서를 일반 GPIO 인 PA4, PA1, PA0 으로 이동
- photo_interrupt_init() 의 포트와 핀 수정
- 판이 막으면 HIGH 라 CLOSED 는 1

| | 센서 핀 (1 층 , 2 층 , 3 층 ) | 코드 | 결과 |
|---|---|---|---|
| Before | PC15, PC14, PC13 | GPIOA 15/14/13 을 읽음 | 센서 값이 바뀌지 않아 층에서 멈추지 않음 |
| After | PA4, PA1, PA0 | GPIOA 4/1/0, CLOSED = 1 | 정상 동작 |

# Conclusion & Discussion

## Conclusion

- 카드 권한 , 마스터 모드 , SCAN 스케줄러를 서비스 모듈로 나눠 구현하고 보드에서 모듈별 시험 통과
- 2-process FSM 과 논블로킹 모터 구동으로 이동 중에도 카드와 GUI 입력 처리
- 8 바이트 UART 프레임과 문 핸드셰이크로 GUI 와 동작을 맞춤

## Discussion

- 모터가 센서를 못 찾았을 때 GUI 에 알리는 프레임이 없음
- 보드 기본 설정에서 쓸 수 없는 핀이 있어 , 핀 배정 전에 보드 매뉴얼 확인 필요
- 초기 계획서에서 기능과 상태 (FSM), 핀 배치를 먼저 정하고 그대로 개발에 적용하니 , 역할 분담과 통합이 수월했고 계획의 중요성을 체감

# Takeaway

## ■ 이기광

- 이전 프로젝트에서는 계획없이 바로 개발에 바로 들어갔을 때 계획이 틀어지고 결과물이 정리가 안되는 경우가 있었음
- 이번 프로젝트에서는 팀원과 개발 계획서를 먼저 작성한 뒤 개발을 시작함
- 그 결과 역할 분담이 명확해졌고 , 구현할 기능과 검증 기준을 미리 정해 진행이 수월했음
- 단순히 기능을 구현하는 것이 아니라 목표를 달성하기 위해 개발한다는 점을 체감함

## ■ 정명훈

- 중심 데이터를 토대로 개발 계획을 세워가는 과정이 길어짐으로써 , 개발 과정이 수월
- 개발하는 시간만큼 디버깅 시간도 길어짐에 따라 , 데이터 시트 확인 및 디버깅 방법이 중요
- Verilog 상의 UART 와 SW 상의 UART 에 대한 이해도가 부족하여 추가 공부가 필요

# Thank You

Q & A